Gerçek VEON Uygulamasına Geçiş Rehberi

Nov 12, 2026 · @cihan demir

Özet ve geçiş sırası

Örnek uygulamadaki desenler agentic-core'a 10 adımda, her biri ayrı PR ve ayrı açma/kapama bayrağıyla taşınır; varsayılan davranış her adımda eski davranıştır. Bu rehber hangi dosyaya ne yazılacağını, nasıl açılacağını, nasıl doğrulanacağını ve nasıl geri alınacağını anlatır. Dosya yolları 09.11.2026 tarihli core snapshot'ına göredir (`src/agentic_core/...`); örnek uygulama yolları `veon-best-practice-sample/backend/app/...` altındadır.

Adım	Ne değişir	Core'da dokunulan yerler	Açma bayrağı	Faz
1	Ortak modüller (loglama, metrik, tracing, Redis araçları, olay yayıncısı) eklenir	Yeni paketler: `observability/`, `redis_kit/`, `events/`	Yok (yalnız kod eklenir)	Faz 1
2	Redis bağlantılarına zaman aşımı, retry, havuz; yetki önbellek anahtarı HMAC	`redis_sentinel.py`, `redis_auth_cache.py`	`REDIS_SOCKET_TIMEOUT` vb., `AUTH_CACHE_KEY_SECRET`	Faz 1
3	Liveness/readiness ayrımı, temiz kapanış	`main.py` (lifespan, `/health`)	Yok	Faz 1
4	Tek satır JSON log, bloklamayan logging, `otel_trace_id`	`main.py` (`_setup_logging`, istek log middleware'i)	`LOG_FORMAT=json` (mevcut)	Faz 2
5	Metriklerde route şablonu, ayrı port, alarmlar	`main.py` (Prometheus middleware, `/metrics`)	`METRICS_PORT`	Faz 2
6	OpenTelemetry tracing	`main.py`, giden HTTP/LLM çağrıları, RabbitMQ başlıkları	`OTEL_SDK_DISABLED=false`	Faz 2
7	Dağıtık rate limit	`api/manage_session_api.py` (4 endpoint)	`RATE_LIMIT_MODE=off/shadow/enforce`	Faz 2
8	Session'a native TTL ve çakışma kontrolü	`redis_session_registry.py`	`SESSION_NATIVE_TTL_ENABLED`, `SESSION_UPDATE_MODE`	Faz 2
9	Olay/log yazımı Redis Stream + worker'a taşınır	`mongo_log_service_client.py` yerine yeni client, worker, migration	`LOG_RESULTS_TYPE=mongo/dual/stream`	Faz 3
10	Helm değişiklikleri ve Redis Sentinel altyapısı	Uygulama chart'ı, Redis release'i	Helm values	Faz 1

Adım 10, diğer adımların ihtiyaç duyduğu values ve secret'ları içerdiği için Faz 1'de başlar ve her fazda genişler.

Ön koşullar ve genel ilkeler

Her değişiklik bayrakla kapalı gelir, önce stage'de açılır ve ölçümle kabul edilir; üretimde açmak bir config değişikliğidir, kapatmak da öyle.

Ön koşullar

- [ ] Stage ortamı üretimle aynı topolojide: en az 2 core replikası, Redis (tercihen Sentinel), MongoDB replica set.
- [ ] Başlangıç ölçümü alındı: p95 gecikme, 5xx oranı, Mongo yazma süresi, Redis hata sayısı, Prometheus seri sayısı. Sonraki adımlar bu değerlerle kıyaslanır.
- [ ] Core'un mevcut `requirements` / `pyproject` sürümleri kontrol edildi (snapshot'ta yok). Hedef: redis-py 6.x, PyMongo 4.13+, OpenTelemetry 1.33.x / 0.54bx (örnek uygulamadaki sabitlemeler).
- [ ] VEON observability bundle kurulu: Alloy OTLP ucu, Loki, Tempo, Prometheus Operator CRD'leri.
- [ ] Yeni secret'lar secret yöneticisinde hazır: `AUTH_CACHE_KEY_SECRET`, gerekiyorsa `REDIS_SENTINEL_PASSWORD`.

İlkeler

- **Küçük PR:** bir adım = bir PR = bir bayrak. İki adım aynı sürümde açılmaz; bir sorun hangi değişiklikten geldiği belli olsun.
- **Varsayılan = eski davranış:** bayrak kapalıyken kod yolu bugünküyle aynı kalır. Davranış değiştiren adımlarda (rate limit, olay hattı) önce `shadow` / `dual` modu çalıştırılır: yeni yol çalışır ve ölçülür ama sonucu kullanılmaz.
- **Kopyala, uyarla:** örnek uygulamadaki dosyalar çalışır ve testlidir; sıfırdan yazılmaz. `from app.` importları `from agentic_core.` olur; ayarlar core'un `settings` nesnesinden (BÜYÜK harfli alanlar) okunur.
- **Testler birlikte gelir:** örnekteki birim testleri ve gerçek Sentinel'e karşı entegrasyon testleri (`tests/test_redis_integration.py`) core'un CI'ına eklenir.
- **Her adımın geri alınması bellidir:** bayrağı kapatmak veya `helm rollback`. Veri taşıyan adımlar (session TTL, olay hattı) için ayrı geri alma adımları "Geri alma planı" bölümündedir.

Adım 1 — Ortak modülleri taşı

Bu adım yalnız kod ekler, hiçbir davranışı değiştirmez; sonraki adımlar bu modülleri kullanır. Dosyalar `veon-best-practice-sample/backend/app/` altından kopyalanır.

Örnek dosya	Core'daki hedef	Uyarlama
`observability/redaction.py`	`agentic_core/observability/redaction.py`	Mevcut `durable_messaging/redaction.py` ile birleştir; tek kaynak kalsın, eski modül yeniyi import etsin
`observability/logging.py`	`agentic_core/observability/logging.py`	`setup_logging` parametrelerini `settings.LOG_LEVEL` vb.'den ver
`observability/metrics.py`	`agentic_core/observability/metrics.py`	Metrik isimleri core ile aynı (`http_requests_total`, `http_request_duration_seconds`); yalnız label'lar değişir
`observability/middleware.py`	`agentic_core/observability/middleware.py`	Legacy `x-message-id` ve `x-trace-id` başlıklarını da loga ekle (Adım 4)
`observability/tracing.py`	`agentic_core/observability/tracing.py`	`Settings` tipi yerine core `settings` nesnesi
`redis_kit/keys.py`, `scripts.py`, `rate_limit.py`	`agentic_core/redis_kit/`	Prefix `vsa` yerine `agentic`
`redis_kit/idempotency.py`, `cache.py`	`agentic_core/redis_kit/` (isteğe bağlı)	Durable messaging zaten idempotency kullanıyor; HTTP katmanında gerekirse
`events/publisher.py`	`agentic_core/events/publisher.py`	Stream anahtarı `agentic:events:logs`
`worker.py`	`agentic_core/events/worker.py`	Adım 9'da `log` / `log_upsert` semantiğine uyarlanır
`migrate.py`	`agentic_core/migrate.py`	`logs` koleksiyonu indeksleri (Adım 9)
`healthcheck.py`	`agentic_core/healthcheck.py`	Değişiklik yok

Bağımlılıklar (core'un requirements dosyasına; sürümler örnekte test edildi):

```
redis==6.2.0
pymongo==4.13.2
prometheus-client==0.22.1
opentelemetry-api==1.33.1
opentelemetry-sdk==1.33.1
opentelemetry-exporter-otlp-proto-http==1.33.1
opentelemetry-instrumentation-fastapi==0.54b1
opentelemetry-instrumentation-redis==0.54b1
opentelemetry-instrumentation-pymongo==0.54b1
opentelemetry-instrumentation-httpx==0.54b1
```

Son satır örnekte yok; core giden HTTP çağrılarında (yetki servisi, data integration, semantic retrieval) iz başlığını taşımak için eklenir. Core'un mevcut redis-py sürümü 6'dan eskiyse önce yükseltme ayrı PR olarak yapılır.

Bitti sayılır: modüller import edilebiliyor, örnekten taşınan birim testleri CI'da yeşil, çalışan davranış değişmedi.

Adım 2 — Redis client'ı sağlamlaştır ve auth cache anahtarını düzelt

Core'daki dört Redis kullanıcısı (session registry, auth cache, division retriever, durable idempotency) client'ı aynı fonksiyondan alıyor: `RedisSentinelConfig.from_env().create_async_client(...)`. Bu yüzden zaman aşımı, retry ve havuz ayarları tek dosyada (`redis_sentinel.py`) eklenir ve dördüne birden uygulanır.

2a. `redis_sentinel.py` — varsayılan bağlantı ayarları

```
from redis.asyncio import BlockingConnectionPool, Redis
from redis.asyncio.retry import Retry
from redis.backoff import ExponentialWithJitterBackoff
from redis.exceptions import ConnectionError as RedisConnectionError

def _defaults() -> dict:
    return {
        "socket_timeout": float(os.getenv("REDIS_SOCKET_TIMEOUT", "0.5")),
        "socket_connect_timeout": float(os.getenv("REDIS_CONNECT_TIMEOUT",
"0.5")),
        "socket_keepalive": True,
        "health_check_interval": int(os.getenv("REDIS_HEALTH_CHECK_INTERVAL",
"15")),
        "max_connections": int(os.getenv("REDIS_MAX_CONNECTIONS", "50")),
        # Yalnız ConnectionError'da tekrar: komut sunucuya ulaşmamıştır,
güvenli.
        # TimeoutError'da tekrar YOK: INCR/XADD/Lua çift çalışabilir.
        "retry": Retry(ExponentialWithJitterBackoff(base=0.05, cap=1.0),
                       int(os.getenv("REDIS_RETRIES", "5")),
                       supported_errors=(RedisConnectionError,)),
        "client_name": f"agentic-core:{socket.gethostname()}",
    }

def create_async_client(self, direct_url, *, decode_responses,
**connection_kwargs):
    kwargs = {**_defaults(), **connection_kwargs, "decode_responses":
decode_responses}
    if not self.enabled:
        pool = BlockingConnectionPool.from_url(direct_url, timeout=1.0,
**kwargs)
        return Redis.from_pool(pool)          # havuz doluysa hata yerine 1
sn bekler
    sentinel = Sentinel(
        self.sentinels,
        min_other_sentinels=1 if len(self.sentinels) > 1 else 0,
```

```
        sentinel_kwargs={"password": self.sentinel_password or None,
                         "socket_timeout": kwargs["socket_timeout"],
                         "socket_connect_timeout":
kwargs["socket_connect_timeout"]},
    )
    kwargs["db"] = _db_from_target(direct_url)
    if self.redis_password:
        kwargs["password"] = self.redis_password
    return sentinel.master_for(self.master_name, **kwargs)
```

Referans: örnekteki `redis_kit/client.py`. Notlar:

- `redis_auth_cache.py` şu an `socket_timeout=5` geçiyor; bu parametreyi silin, ortak 0,5 sn varsayılanı kullanılsın.
- Engelleyen (blocking) komut kullanan yeni kod (Adım 9 worker'ı) `socket_timeout`'u blok süresinden uzun geçmelidir.
- redis-py 6'dan eskiyse `ExponentialWithJitterBackoff` yerine `EqualJitterBackoff` kullanın.

2b. `redis_auth_cache.py` — anahtarı HMAC ile üret

```
import hashlib, hmac

def _make_key(self, division_id: str, token: str) -> str:
    digest = hmac.new(self._key_secret, token.encode(),
hashlib.sha256).hexdigest()[:32]
    return f"{self.key_prefix}{division_id}:{digest}"
```

- `self._key_secret = os.environ["AUTH_CACHE_KEY_SECRET"].encode()` — Kubernetes Secret'tan gelir; tüm pod'larda aynı olmalı.
- Log çağrılarındaki `"key": key` alanını kaldırın (division id ve token özeti loga gitmesin).
- Geçiş etkisi: eski `hash()` anahtarları bir daha okunmaz ve TTL (300 sn) dolunca kendiliğinden silinir; ilk 5 dakika önbellek soğuk çalışır.

Doğrulama: stage'de `make failover-freeze` eşdeğeri (master'ı dondurma) sırasında istekler 0,5 sn içinde hata dönüp failover sonrası kendiliğinden düzelmeli; `CLIENT LIST` çıktısında `agentic-core:<pod>` isimleri görünmeli; iki pod'a aynı token ile gelen ikinci istek auth cache'ten dönmeli.

Adım 3 — Sağlık kontrolleri ve temiz kapanış

Bugün `/health` her zaman `ok` dönüyor ve kapanışta `await mongodb_client.close()` senkron bir metodu beklediği için hata veriyor. Bu adımda üç şey yapılır.

3a. Kapanış hatası (`main.py` satır ~583)

Motor'un `close()` metodu senkron olduğu için şimdilik `await` kaldırılır: `mongodb_client.close()`. PyMongo Async'e geçilince (Adım 9 sonrası) `await client.close()` olur.

3b. Liveness ve readiness ayrımı

Uç	Kontrol eder	Kontrol etmez	Kubernetes probe
`/healthz`	Process ayakta mı	Bağımlılık (Redis/Mongo düştü diye pod'ları yeniden başlatmak restart fırtınası yaratır)	startup + liveness
`/readyz`	MongoDB ping (1 sn zaman aşımı)	Redis: yalnız durum olarak raporlanır, 503 üretmez	readiness
`/health`	`/healthz` ile aynı	—	Mevcut probe ve monitorler bozulmasın diye bir süre takma ad olarak kalır

Kod: örnekteki `api/system.py` içindeki `healthz`, `readyz`, `_mongo_ping`, `_redis_ping` fonksiyonları. Sağlık uçları tracing ve access log dışında tutulur.

3c. Kapanış sırası (lifespan `finally`)

1. uvicorn SIGTERM'de yeni bağlantı almayı keser, açık istekleri bitirir (`--timeout-graceful-shutdown 20`).
2. Olay yayıncısı bellek kuyruğunu boşaltır (Adım 9 sonrası, 5 sn sınır).
3. Redis ve Mongo client'ları kapatılır.
4. Tracing kuyruğu gönderilir (`force_flush` 2 sn).

Helm tarafında: startup probe `/healthz` (`failureThreshold: 30`, `periodSeconds: 2`), liveness `/healthz`, readiness `/readyz`, `preStop: sleep 5`, `terminationGracePeriodSeconds: 30`. Referans: örnek chart'taki `templates/backend.yaml`.

Doğrulama: trafik altında rolling update sırasında 5xx sıfır; Mongo erişilemezken pod'lar `NotReady` olur ama yeniden başlamaz; pod loglarında kapanış hatası yok.

Adım 4 — Yapılandırılmış ve bloklamayan loglama

Core zaten JSON log üretiyor; eksik olanlar bloklamayan yazım, `otel_trace_id` alanı, istek başına tek satır ve kişisel verinin dışarıda tutulması. Uygulama yine yalnız stdout'a yazar; logları kümedeki Alloy (veya Fluent Bit) taşır.

4a. `main.py` — `_setup_logging` yerine ortak kurulum

```
from agentic_core.observability.logging import setup_logging

if settings.ENABLE_LOGGING:
    setup_logging(level=settings.LOG_LEVEL, service="agentic-core",
                  version=os.getenv("APP_VERSION", "dev"),
                  environment=settings.ENVIRONMENT,
                  queue_size=int(os.getenv("LOG_QUEUE_SIZE", "10000")))
```

Bu kurulum: sınırlı kuyruk + ayrı yazma thread'i (stdout yavaşlasa da istek beklemez; kuyruk dolarsa kayıt düşer ve `app_logs_dropped_total` artar), her satıra `otel_trace_id`, `otel_span_id`, `request_id`, ve maskeleme. `LOG_FORMAT=text` yerel geliştirme için korunabilir.

4b. `main.py` — istek log middleware'ini değiştir

`_request_logging_middleware` (başladı + bitti, iki satır) kaldırılır; yerine `app.add_middleware(ObservabilityMiddleware)` eklenir. Core'a özgü iki uyarlama:

- Legacy başlıklar korunur: `x-message-id` → `message_id`, `x-trace-id` → `trace_id` alanı olarak access log'a eklenir. VEON README'ye göre bu iş trace id'si OTel'den bağımsızdır; mevcut Loki sorguları bozulmaz.
- `query_params`, `client_ip` ve `user_agent` loglanmaz. Güvenlik denetimi için IP gerekiyorsa HMAC özeti yazılır.

4c. Uygulama içi log çağrılarını tara

```
grep -rnE 'logger\.(info|warning|error|debug)\(f"' src/agentic_core | grep -
iE 'body|prompt|payload|token|request|response'
```

Bulunan satırlar sabit mesaj + `extra={...}` biçimine çevrilir; prompt, yanıt, request/response gövdesi ve token hiç loglanmaz.

4d. Altyapı

- uvicorn `--no-access-log` ile çalışır (access log'u middleware yazar).
- Core namespace'i Alloy'un `alloy.logNamespaces` listesinde olmalı ve aynı namespace'i başka toplayıcı izlememeli (çift log).

- Grafana Loki datasource'unda `otel_trace_id` derived field'ı tanımlı olmalı (VEON README adım 7).

Doğrulama: `{namespace="veon-ns"} | json | otel_trace_id != ""` sorgusu core satırlarını döndürmeli; log satırındaki bağlantıdan Tempo'ya geçilebilmeli; e-posta ve `Bearer` araması boş dönmeli; istek başına log satırı sayısı yarıya inmeli.

Adım 5 — Metrikler, alarmlar, dashboard

Metrik isimleri aynı kalır; değişen şey label'lardır: ham `path` yerine route şablonu. Bu, `/sessions/<id>` gibi her benzersiz URL'nin yeni zaman serisi açmasını durdurur.

5a. `main.py` — Prometheus middleware'i

`_prom_http_middleware` kaldırılır; Adım 4'te eklenen `ObservabilityMiddleware` aynı metrikleri `route` label'ıyla yazar (`scope["route"].path` , eşleşmeyen istekler `__unmatched__`).

Geçiş uyumluluğu: mevcut dashboard ve alarmlar `path` , `service` veya `env` label'ını kullanıyorsa önce onları bulun:

```
grep -rnE 'http_requests_total|http_request_duration_seconds' <dashboard-ve-alarm-reposu>
```

Kullanılıyorsa bir sürüm boyunca `service` ve `env` label'ları korunur, sorgular `route` 'a geçirildikten sonra kaldırılır. `service` / `env` bilgisi zaten scrape tarafından (`job` , `namespace`) gelir.

5b. Ayrı metrik portu, tek process

- `/metrics` 8000 yerine 9100 portunda yayınlanır: lifespan'de `start_metrics_server(int(os.getenv("METRICS_PORT", "9100")))` . Ingress bu portu açmaz; NetworkPolicy yalnız `observability` namespace'ine izin verir. Bu sayede `METRICS_TOKEN` gerekmez.
- Pod başına tek uvicorn process'i kullanılır (`--workers` yok); `PROMETHEUS_MULTIPROC_DIR` kaldırılır. Ölçek HPA ile.
- Geçiş sırasında eski `/metrics` ucu bir sürüm daha 8000'de kalabilir; ServiceMonitor `metrics` portuna geçirilince silinir.

5c. Yeni metrikler

Metrik	Nereden	Ne için
`redis_errors_total{operation,error}`	Redis çağrı noktaları	Redis sorunlarını görmek

Metrik	Nereden	Ne için
`cache_requests_total{cache="auth",result}`	`redis_auth_cache.py` get	Auth cache isabet oranı (Adım 2 doğrulaması)
`rate_limit_decisions_total{limiter,decision}`	Adım 7	Limit ayarı ve kötüye kullanım
`session_update_conflicts_total`	Adım 8	Eşzamanlı güncelleme sıklığı
`events_*`, `events_stream_lag`	Adım 9	Olay hattı sağlığı
`app_logs_dropped_total`	Adım 4	Log kaybı

Histogram gözlemlerine exemplar (trace id) eklenir; Prometheus CR'ında `enableFeatures: [exemplar-storage]` açılır.

5d. Alarm ve dashboard

- Örnek chart'taki `templates/observability.yaml` içindeki PrometheusRule core'un job adlarıyla (`<release>-agentic-core`, `<release>-agentic-core-worker`) uyarlanıp uygulama chart'ına eklenir.
- Örnek dashboard (`files/veon-sample-overview.json`) datasource değişkenleri kullandığı için core'a doğrudan uyar; `app_info` metriği core'a eklenince job filtreleri otomatik dolar.

Doğrulama: `count({__name__=~"http_request.*"})` seri sayısı başlangıç ölçümüne göre belirgin düşmeli; Grafana gecikme grafiğindeki exemplar noktasından Tempo'ya geçilebilmeli; stage'de 5xx üretilerek alarmın tetiklendiği görülmeli.

Adım 6 — Tracing (OpenTelemetry) aktivasyonu

Amaç: tek bir `process_inquiry` isteğinin yetki kontrolü, session okuma, LLM çağrısı ve olay yazımıyla birlikte tek trace'te görünmesi. Uygulama yalnız Alloy'un OTLP ucunu bilir; Tempo/Jaeger seçimi collector'dadır.

Önce kontrol edin: VEON observability README'si core için SDK içeren ayrı bir imaj hattından (`global.tracing`, `agentic-tracing-config` ConfigMap'i) söz ediyor. Core'un geliştirme dalında bu SDK kurulumu zaten varsa örnekteki `tracing.py` onun yerine değil, eksiklerini tamamlamak için kullanılır (iki ayrı provider kurulmaz).

6a. Kurulum (`main.py`, app oluşturulmadan önce)

```
from agentic_core.observability.tracing import setup_tracing
tracer_provider = setup_tracing(settings_adapter,
service_name=os.getenv("OTEL_SERVICE_NAME", "agentic-core"))
...
if tracer_provider:
    FastAPIInstrumentor.instrument_app(app, tracer_provider=tracer_provider,
        excluded_urls="health,healthz,readyz,metrics", exclude_spans=
["receive", "send"])
    HTTPXClientInstrumentor().instrument(tracer_provider=tracer_provider)
```

`OTEL_SDK_DISABLED=true` iken hiçbir şey kurulmaz. Kuyruk sınırlıdır (2048 span); collector erişilemezse span düşer, istek etkilenmez.

6b. Otomatik instrumentation kapsamı

Kaynak	Instrumentor	Not
Gelen HTTP	FastAPI	Route şablonuyla span adı
Redis (4 client)	Redis	Lua `EVALSHA` olarak görünür; polling döngüleri `untraced()` içinde
MongoDB	PyMongo	Motor thread havuzu kullandığı için bağlam kopabilir; tam sonuç PyMongo Async geçişinden sonra
Giden HTTP (yetki, data integration, semantic retrieval)	HTTPX	`traceparent` başlığı alt servislere taşınır

6c. Elle eklenecek span'ler

- Controller adımları ve LLM çağrıları: `tracer.start_as_current_span("llm.chat", attributes={"coform.operation": "llm.chat", "gen_ai.operation.name": "chat", "gen_ai.request.model": model, "gen_ai.usage.input_tokens": n_in, "gen_ai.usage.output_tokens": n_out})`. VEON Alloy'u yalnız izin listesindeki alanları (`coform.operation`, `gen_ai.*`, `http.*`, `messaging.*`, `error.type`) taşır; prompt ve yanıt hiçbir zaman attribute olmaz.
- RabbitMQ (durable messaging): yayınlarken `TraceContextTextMapPropagator().inject(headers)` ile mesaj başlıklarına `traceparent` eklenir; worker'da `extract(message.headers)` ile `SpanKind.CONSUMER` span'i açılır. Böylece HTTP → kuyruk → worker zinciri tek trace olur (örnekteki `events/publisher.py` ve `worker.py` ile aynı desen).

6d. Açma

Helm: `global.tracing.enabled=true`, `endpoint=http://veon-observability-alloy.observability.svc.cluster.local:4318`, `samplingRatio="0.1"` (stage pilotunda `"1"`). Ortam değişkenleri: `OTEL_SDK_DISABLED=false`, `OTEL_EXPORTER_OTLP_ENDPOINT`, `OTEL_TRACES_SAMPLER_ARG`, `OTEL_SERVICE_NAME`, `APP_VERSION`, `DEPLOYMENT_ENVIRONMENT`.

Doğrulama: stage'de bir `process_inquiry` isteğinin trace'inde HTTP, yetki servisi çağrısı, Redis, LLM ve (Adım 9 sonrası) worker span'leri tek trace id ile görünür; span attribute'larında prompt veya token yoktur; collector durdurulunca API yanıtları değişmez.

Adım 7 — Rate limiting

Rate limit, kimliği doğrulanmış isteğe uygulanır ve sayacı Redis'te tutar; böylece tüm core replikaları aynı limiti paylaşır. LLM maliyeti ve backend yükü için en değerli koruma budur.

7a. Nereye

`api/manage_session_api.py` içinde dört endpoint `verify_division_access(request, req.division_id)` çağırıyor: `create_session` (satır ~316), `process_inquiry` (~383), `trigger_automation` (~537), `continue_agent_loop` (~686). Limit, bu çağrının hemen arkasına eklenir; yetkisiz istek limit sayacını tüketmez.

```
from agentic_core.redis_kit.rate_limit import enforce_rate_limit

await verify_division_access(request, req.division_id)
await enforce_rate_limit(request, endpoint="process_inquiry",
division_id=req.division_id)
```

7b. `enforce_rate_limit` yardımcısı (örnekteki `SlidingWindowRateLimiter` üzerine)

```
async def enforce_rate_limit(request, *, endpoint: str, division_id: str) ->
None:
    mode = os.getenv("RATE_LIMIT_MODE", "off")        # off | shadow |
enforce
    if mode == "off":
        return
    limiter = request.app.state.rate_limiters[endpoint]
    token = request.headers.get("authorization", "")
    identity = f"{division_id}:{hmac_digest(token)}"   # kullanıcı + division
    result = await limiter.hit(identity)
    if not result.allowed and mode == "enforce":
        raise HTTPException(429, detail={"code": "rate_limited"},
                            headers={"Retry-After": str(max(1,
result.retry_after_ms // 1000))})
```

- Kimlik: istemcinin gönderdiği başlık değil, doğrulanmış token'ın HMAC özeti + division. IP tek başına kullanılmaz (NAT arkasındaki kurumlar aynı IP'yi paylaşır).
- Limitler ayardan okunur, örn. `RATE_LIMITS={"process_inquiry":"60/60","trigger_automation":"30/60","continue_agent_loop":"120/60","create_session":"20/60"}` (istek/saniye penceresi). Değerler örnektir; shadow verisiyle belirlenir.
- Redis hatasında fail-open: istek geçer, `rate_limit_decisions_total{decision="fail_open"}` artar.
- Limiterler lifespan'de bir kez kurulur (`app.state.rate_limiters`); anahtar öneki `agentic:rl:<endpoint>:<özet>`, TTL pencere kadar.

7c. Açma sırası

1. `RATE_LIMIT_MODE=shadow`: karar verilir ve metriğe yazılır, istek engellenmez. En az bir hafta çalışır.
2. `rate_limit_decisions_total{decision="limited"}` division bazında incelenir; meşru kullanımı kesmeyecek limitler seçilir.
3. `RATE_LIMIT_MODE=enforce` önce stage'de, sonra üretimde açılır. İstemciler (Dashboard, entegrasyonlar) 429 + `Retry-After`'a göre tekrar denemeli; bu istemci ekiplerine önceden duyurulur.

Doğrulama: iki replikaya dağılan istekte limit tam uygulanır (örnekteki ölçüm: 30 istekte tam 10 izin); Redis durdurulunca istekler geçmeye devam eder ve fail-open metriği artar.

Adım 8 — Session: native TTL ve atomik güncelleme

Bu adım iki sorunu kapatır: hiç okunmayan süresi dolmuş session'ların Redis'te sonsuza kadar kalması ve aynı session'a eşzamanlı iki güncellemede birinin diğerini sessizce ezmesi. Dosya: `registries/session_registries/redis_session_registry.py`.

8a. Native TTL (`SESSION_NATIVE_TTL_ENABLED=true`)

- `create_session`: `await self._redis.set(key, data, nx=True, ex=ttl)`
- `REPLACE_EXPIRED_SESSION_SCRIPT`: `redis.call('SET', KEYS[1], ARGV[2], 'EX', ARGV[3])` — TTL argümanı eklenir.
- `update_session_data`: her yazımda `ex=ttl` (kayan süre; mevcut `refresh_session_expiry` mantığıyla aynı).
- `expires_at` alanı uygulama mantığı için kalır; Redis TTL'i bundan birkaç dakika uzun tutulabilir ki artifact temizliği (`cleanup_session_artifacts`) okuma sırasında çalışmaya devam etsin.

Mevcut TTL'siz anahtarlar için tek seferlik betik (session DB'si, `SCAN` ile; `KEYS` kullanılmaz):

```
async for key in redis.scan_iter(match="session:*", count=500):
    if await redis.ttl(key) != -1:          # zaten TTL'li
        continue
    raw = await redis.get(key)
    expires_at = orjson.loads(raw).get("expires_at") if raw else None
    if expires_at and expires_at > time.time():
        await redis.expireat(key, int(expires_at) + 300)
    else:
        await redis.expire(key, 3600)        # süresi dolmuş: 1 saat sonra
silinsin
```

8b. Çakışma kontrolü (`SESSION_UPDATE_MODE=legacy|detect|cas`)

Session JSON'una `_version` sayısı eklenir; `get_session_data` bunu döner, güncelleme okunan sürümü karşılaştırır. Redis `WATCH/MULTI/EXEC` ile oku-yaz arasına başka yazım girerse işlem iptal olur.

```
async with self._redis.pipeline(transaction=True) as pipe:
    await pipe.watch(key)
    existing = orjson.loads(await pipe.get(key))
    if session_data.get("_version") != existing.get("_version"):
        SESSION_CONFLICTS.inc()
        if mode == "cas":
            raise SessionUpdateConflict(session_id)
    merged = self.preserve_immutable_session_fields(existing, session_data)
    merged["_version"] = existing.get("_version", 0) + 1
    pipe.multi()
    pipe.set(key, orjson.dumps(self.refresh_session_expiry(merged)), ex=ttl)
    await pipe.execute()          # araya yazım girdiyse WatchError → çakışma
```

Önemli: `update_session_data` çağırandan **tam session** alıyor, değişiklik farkını değil. Bu yüzden çakışmada sessizce tekrar denemek yine ezme yapar; doğru çözüm çağıran adımın session'ı yeniden okuyup değişikliğini tekrar uygulamasıdır. Durable messaging'in session başına sıra garantisi kullanılan yollarda çakışma zaten nadirdir.

Açma sırası: `detect` modunda bir hafta çalıştır, `session_update_conflicts_total` ile gerçek çakışma sıklığını ölç; sıfır değilse çakışan controller yollarını yeniden oku-uygula döngüsüyle güncelle, sonra `cas`'a geç.

Doğrulama: `redis-cli --scan --pattern 'session:*'` ile örneklenen anahtarların `TTL` değeri -1 değil; Redis bellek kullanımı sabitleniyor; aynı session'a paralel iki güncelleme testinde biri çakışma alıyor ve veri kaybolmuyor.

Adım 9 — Olay/log yazımını asenkron hatta taşı

Core, olayları `log_service_client.log_upsert(...)` ile dört noktada istek içinde MongoDB'ye yazıyor: `api/manage_session_api.py` (~92, ~497) ve `durable_messaging/execution_service.py` (~279, ~419 — akış parça logları). Aynı arayüzü koruyan yeni bir client yazılırsa bu çağrı noktalarının hiçbiri değişmez.

Ön koşul: stream'in tutulduğu Redis `maxmemory-policy noeviction` olmalıdır. Mevcut Redis önbellek için `allkeys-lru` ile çalışıyorsa stream kayıtları sessizce silinebilir; bu durumda stream için ayrı Redis kurulur (Adım 10).

9a. Aynı arayüzle yeni client (`clients/log_service_clients/stream_log_service_client.py`)

```
class StreamLogServiceClient:
    """MongoLogServiceClient ile aynı arayüz; I/O yapmaz, beklemez."""
    def __init__(self, publisher: EventPublisher):
        self._publisher = publisher

    async def log(self, log_entry: dict) -> None:
        self._publisher.publish("log", redact_sensitive(log_entry))

    async def log_upsert(self, log_entry: dict, *, event_id: str) -> None:
        doc = {**redact_sensitive(log_entry), "event_id": event_id, "_seq":
time.time_ns()}
        self._publisher.publish("log_upsert", doc, event_id=f"{event_id}:
{doc['_seq']}")

class DualLogServiceClient:
    """Gölge mod: bugünkü Mongo yazımı + stream (worker logs_shadow'a
yazar)."""
    def __init__(self, mongo, stream): self._mongo, self._stream = mongo,
stream
    async def log(self, e): await self._mongo.log(e); await
self._stream.log(e)
    async def log_upsert(self, e, *, event_id):
        await self._mongo.log_upsert(e, event_id=event_id)
        await self._stream.log_upsert(e, event_id=event_id)
```

`main.py` satır ~545'teki seçim genişler: `LOG_RESULTS_TYPE` = `none` | `mongo` (bugünkü) | `dual` (gölge) | `stream` (hedef). Aynı client nesnesi durable messaging'in `CommandExecutionService`'ine de verilir. Publisher lifespan'de başlatılır, kapanışta kuyruğu boşaltır (Adım 3c).

9b. Worker: `log_upsert` sırasını koru

Core'un `log_upsert`'ü aynı `event_id`'yi güncelleyerek yaşam döngüsü durumunu (örn. received → completed) yazıyor. Asenkron hatta mesajlar sırasız gelebilir; eski sürümün yenisini ezmemesi için `_seq` koşulu kullanılır:

```
ReplaceOne(
    {"event_id": doc["event_id"], "$or": [{"_seq": {"$lt": doc["_seq"]}},
{"_seq": {"$exists": False}}]},
    doc, upsert=True)
# Daha yeni sürüm varsa filtre eşleşmez → upsert insert dener → E11000 →
"eski sürüm", yok sayılır.
```

`log` (tekil kayıt) mesajları örnekteki gibi `$setOnInsert` ile yazılır. Worker ayrı bir Deployment'tır (`python -m agentic_core.events.worker`, aynı imaj, 2 replika). Hedef koleksiyon `WORKER_TARGET_COLLECTION` ile seçilir: gölge modda `logs_shadow`, sonra `logs`.

9c. Migration: `logs.event_id` benzersiz indeksi

Mevcut çift kayıtlar indeks oluşturmayı durdurur; önce bulunur ve temizlenir:

```
db.logs.aggregate([
  {$match: {event_id: {$exists: true}}},
  {$group: {_id: "$event_id", n: {$sum: 1}, ids: {$push: "$_id"}}},
  {$match: {n: {$gt: 1}}}
], {allowDiskUse: true})
// Her grupta en güncel kayıt kalır, diğerleri silinir (önce yedek!)
db.logs.createIndex({event_id: 1}, {unique: true, name: "uniq_event_id",
  partialFilterExpression: {event_id: {$exists: true}}})
```

İndeks, Helm pre-upgrade hook Job'ındaki `python -m agentic_core.migrate` ile ve düşük trafikte oluşturulur. Saklama süresi (TTL indeksi) ayrı bir karardır; otomatik eklenmez.

9d. Geçiş sırası

1. Worker + migration deploy edilir; `LOG_RESULTS_TYPE=dual`, worker `logs_shadow`'a yazar.
2. En az 7 gün boyunca saatlik kayıt sayıları `logs` ile `logs_shadow` arasında karşılaştırılır (event_family bazında); `events_dropped_total` = 0, `events_stream_lag` düşük.
3. `LOG_RESULTS_TYPE=stream`, worker hedefi `logs`. API p95'inde Mongo yazma süresi kadar düşüş beklenir.
4. Bir hafta sonra `logs_shadow` silinir.

Garanti ve sınır: at-least-once teslim + benzersiz indeks = her olay bir kez. Pod ani çökerse bellek kuyruğundaki (1 sn'den kısa) olaylar kaybolabilir. Kayıp hiç kabul edilemiyorsa Redis Stream yerine mevcut RabbitMQ (quorum queue + publisher confirm) kullanılır; arayüz aynı kalır.

Doğrulama: worker'lar durdurulup trafik gönderildiğinde API gecikmesi değişmez, worker'lar dönünce birikim erir; Mongo yavaşlatıldığında (örn. `tc` ile gecikme) API p95 sabit kalır; aynı olay iki kez yayınlandığında Mongo'da tek kayıt olur.

Adım 10 — Helm/Kubernetes ve Redis HA altyapısı

Uygulama chart'ı bu pakette yok; aşağıdaki parçalar örnek chart'tan (`deploy/helm/veon-sample-app/templates/`) core chart'ına taşınır. Her parça ilgili adımla aynı sürümde gelir.

Chart parçası	Örnekteki karşılığı	Hangi adım
Yeni secret anahtarları (`AUTH_CACHE_KEY_SECRET` , `REDIS_SENTINEL_PASSWORD`) — yalnız `secretKeyRef`	`_helpers.tpl` → `vsa.secretEnv`	2
Startup/liveness/readiness probe'ları, `preStop: sleep 5` , `terminationGracePeriodSeconds`	`backend.yaml`	3
`metrics` portu (9100), ServiceMonitor'un bu porta geçirilmesi, NetworkPolicy	`backend.yaml` , `observability.yaml` , `networkpolicy.yaml`	5
PrometheusRule ve dashboard ConfigMap'i	`observability.yaml` , `files/veon-sample-overview.json`	5
`global.tracing` → tracing ConfigMap'i + checksum annotation	`tracing-configmap.yaml` , `vsa.checksums`	6
`RATE_LIMIT_MODE` , `RATE_LIMITS` ConfigMap değerleri	`configmap.yaml`	7
Event worker Deployment'ı, metrik Service'i, PDB, (isteğe bağlı) KEDA ScaledObject	`worker.yaml` , `pdb.yaml`	9
Migration hook Job'ı (`pre-install,pre-upgrade`)	`migration-job.yaml`	9

Chart parçası	Örnekteki karşılığı	Hangi adım
`topologySpreadConstraints`, `readOnlyRootFilesystem`, `drop: [ALL]`, `automountServiceAccountToken: false`	`backend.yaml`	Herhangi bir sürümde

Redis Sentinel'e geçiş (Redis şu an `REDIS_MODE=standalone` ise)

1. Sentinel kümesi kurulur: 3 düğüm, her pod'da redis + sentinel, ayrı node'lar, AOF, `min-replicas-to-write 1`. Örnek values: `deploy/helm/examples/redis-sentinel.values.yaml` (imaj kaynağı kurum içinde onaylanmalı).
2. Veri taşıma: yeni master geçici olarak eski standalone'un replikası yapılır (`REPLICAOF <eski> 6379`), senkron olunca `REPLICAOF NO ONE`. Böylece aktif session'lar kaybolmaz. Session kaybı kabul edilebiliyorsa bakım penceresinde doğrudan geçilebilir.
3. Core env: `REDIS_MODE=sentinel`, `REDIS_SENTINELS=["redis-node-0.redis-headless.redis.svc.cluster.local:26379", ...]`, `REDIS_SENTINEL_MASTER_NAME=mymaster`. Mevcut `REDIS_URL` + DB index deseni Sentinel'de çalışmaya devam eder.
4. Karar: önbellek (auth cache) ve kalıcı veriler (session, stream, idempotency) için ayrı Redis kurulumları. Ayrılırsa önbellek kurulumu `allkeys-lru` ve AOF'suz çalışır.

Doğrulama: `helm lint`, `helm template | kubeconform -strict` (CRD şemalarıyla) ve `helm upgrade --dry-run=server` temiz; stage'de Sentinel tatbikatları (Doğrulama bölümü) geçti.

Doğrulama: kabul testleri ve tatbikatlar

Bir adım, aşağıdaki ilgili maddeler stage'de işaretlenmeden üretimde açılmaz. Örnek uygulamadaki `make` hedefleri aynı senaryoları yerelde gösterir.

Sürekli (CI)

- [ ] Örnekten taşınan birim testleri ve gerçek Sentinel'e karşı entegrasyon testleri (flash-sale yerine session/rate limit/idempotency senaryoları) yeşil.
- [ ] `helm lint`, `kubeconform -strict`, `helm upgrade --dry-run=server` temiz.

Redis (Adım 2, 10)

- [ ] Kontrollü failover (`SENTINEL FAILOVER`): yazma kesintisi ölçüldü (örnekte ~9 sn `NOREPLICAS`), sonrasında hata yok.

- [ ] Master pod'u silindi: kesinti ölçüldü (örnekte ~2 sn), elle müdahale gerekmedi.
- [ ] Master ağı kesildi (NetworkPolicy ile izole): istekler 0,5 sn içinde hata döndü, asılı kalmadı.
- [ ] Eski master geri geldiğinde replika olarak katıldı.

Loglama, metrik, tracing (Adım 4–6)

- [ ] Core log satırlarında `otel_trace_id` var; Loki'den Tempo'ya geçiş çalışıyor.
- [ ] E-posta, `Bearer`, `password=` araması son 24 saatte boş.
- [ ] HTTP metrik seri sayısı başlangıç ölçümüne göre düştü; alarmlar stage'de bilinçli 5xx ile tetiklendi.
- [ ] Bir `process_inquiry` trace'i HTTP, yetki, Redis, LLM ve worker span'lerini tek trace id'de gösteriyor; attribute'larda prompt yok.
- [ ] Collector durdurulunca API yanıtları ve gecikmesi değişmedi.

Rate limit ve session (Adım 7–8)

- [ ] Shadow verisiyle limitler belirlendi; enforce sonrası meşru trafikte 429 oranı kabul edilebilir.
- [ ] Redis durdurulunca rate limit fail-open çalıştı.
- [ ] Aynı session'a paralel güncelleme testi: çakışma algılandı, veri kaybolmadı.
- [ ] TTL'siz session anahtarı kalmadı; Redis bellek kullanımı sabit.

Olay hattı (Adım 9)

- [ ] Gölge modda 7 gün boyunca `logs` ve `logs_shadow` sayıları eşleşti; `events_dropped_total` = 0.
- [ ] Worker'lar durdurulunca API gecikmesi değişmedi; geri gelince birikim eridi.
- [ ] Mongo yavaşlatıldığında API p95 sabit kaldı.
- [ ] Aynı olay iki kez yayınlandığında Mongo'da tek kayıt; eski `log_upsert` sürümü yenisini ezmedi.

Dağıtım (Adım 3)

- [ ] Trafik altında rolling update: 5xx sıfır, kapanış hatası yok, worker ACK'lenmemiş mesaj bırakmadı.

Geri alma planı

Adımların çoğu bir bayrağı kapatarak veya `helm rollback` ile dakikalar içinde geri alınır; veri taşıyan iki adım (session ve olay hattı) için sıra önemlidir.

Adım	Geri alma yolu	Veri etkisi
2 Redis client	`REDIS_SOCKET_TIMEOUT` vb. artırılır veya `helm rollback`	Yok
2 Auth cache HMAC	`helm rollback`	Yok; önbellek bir kez soğur (en fazla 300 sn)
3 Sağlık ve kapanış	`helm rollback`; `/health` takma adı zaten duruyor	Yok
4–5 Loglama ve metrik	`helm rollback`; eski dashboard sorguları bir sürüm boyunca çalışmaya devam eder	Yok
6 Tracing	`global.tracing.enabled=false` (yalnız pod yeniden başlar)	Yok
7 Rate limit	`RATE_LIMIT_MODE=shadow` veya `off`	Yok
8 Session TTL	`SESSION_NATIVE_TTL_ENABLED=false`; TTL'li anahtarlar geçerli kalır	Yok; eski kod `_version` alanını yok sayar
8 Session çakışma	`SESSION_UPDATE_MODE=detect` veya `legacy`	Yok
9 Olay hattı	1) `LOG_RESULTS_TYPE=mongo` (yeni olaylar yeniden istek içinde yazılır) 2) worker `events_stream_lag` ve pending 0 olana kadar çalışmaya devam eder 3) sonra worker kapatılır	Worker boşaltılmadan kapatılırsa stream'deki olaylar Mongo'ya yazılmaz
10 Redis Sentinel	`REDIS_MODE=standalone` + `REDIS_URL` master servisine; veya eski standalone'a geri dönüş	Geçişten sonra yazılan session'lar eski kurulumda yoktur

Migration Job'ın oluşturduğu benzersiz indeks geri alınmaz. Eski kodun `replace_one({"event_id": ...}, upsert=True)` çağrılarıyla uyumludur (MongoDB 4.2+ eşitlik filtreli upsert çakışmalarını sunucuda yeniden dener); aynı `event_id` ile ikinci bir kayıt eklenmeye çalışılırsa artık hata alınır, bu da istenen davranıştır. Her geri almadan sonra ilgili kabul maddesi yeniden çalıştırılır.

Ek: yeni ortam değişkenleri

Aşağıdaki isimler öneridir; core'daki mevcut adlandırmaya göre değiştirilebilir. Gizli değerler yalnız Kubernetes Secret'tan gelir.

Değişken	Varsayılan	Açıklama	Adım
`REDIS_SOCKET_TIMEOUT`	`0.5`	Redis komut zaman aşımı (sn)	2
`REDIS_CONNECT_TIMEOUT`	`0.5`	Redis bağlantı zaman aşımı (sn)	2
`REDIS_MAX_CONNECTIONS`	`50`	Client başına havuz üst sınırı	2
`REDIS_RETRIES`	`5`	Bağlantı hatasında tekrar sayısı	2
`REDIS_HEALTH_CHECK_INTERVAL`	`15`	Boştaki bağlantı kontrolü (sn)	2
`AUTH_CACHE_KEY_SECRET`	— (Secret)	Auth cache anahtarı için HMAC sırrı	2
`LOG_QUEUE_SIZE`	`10000`	Log kuyruğu kapasitesi	4
`METRICS_PORT`	`9100`	Ayrı metrik portu	5
`OTEL_SDK_DISABLED`	`true`	Tracing kapalı/açık	6
`OTEL_EXPORTER_OTLP_ENDPOINT`	VEON Alloy `:4318`	OTLP hedefi	6
`OTEL_TRACES_SAMPLER_ARG`	`0.1`	Örnekleme oranı	6
`OTEL_SERVICE_NAME`	`agentic-core`	Servis adı (worker için ayrı)	6
`APP_VERSION`	imaj etiketi	Sürüm bilgisi (log, trace, `app_info`)	4–6
`RATE_LIMIT_MODE`	`off`	`off` / `shadow` / `enforce`	7

Değişken	Varsayılan	Açıklama	Adım
`RATE_LIMITS`	JSON	Endpoint başına `istek/saniye` penceresi	7
`SESSION_NATIVE_TTL_ENABLED`	`false`	Session anahtarlarına Redis TTL'i	8
`SESSION_UPDATE_MODE`	`legacy`	`legacy` / `detect` / `cas`	8
`LOG_RESULTS_TYPE`	`none` (mevcut)	Yeni değerler: `dual`, `stream`	9
`EVENTS_STREAM_KEY`	`agentic:events:logs`	Olay stream anahtarı	9
`EVENTS_STREAM_MAXLEN`	`100000`	Stream üst sınırı (yaklaşık)	9
`EVENTS_QUEUE_SIZE`	`10000`	Bellek kuyruğu kapasitesi	9
`WORKER_TARGET_COLLECTION`	`logs`	Gölge modda `logs_shadow`	9
`WORKER_GROUP`, `WORKER_BATCH_SIZE`, `WORKER_CLAIM_IDLE_MS`, `WORKER_MAX_DELIVERIES`	`audit-persister`, `200`, `30000`, `5`	Worker ayarları	9
`MONGO_TIMEOUT_MS`, `MONGO_MAX_POOL_SIZE`	`3000`, `50`	MongoDB operasyon zaman aşımı ve havuz	9 (PyMongo Async ile)

Kaldırılacaklar: `PROMETHEUS_MULTIPROC_DIR` (Adım 5), `METRICS_TOKEN` (metrik portu NetworkPolicy ile korununca).